

딥러닝 파이토치 교과서 6.1.5

13.1. 생성 모델이란

13.1.1. 생성 모델 개념

- **생성 모델**
 - 주어진 데이터의 분포와 특성을 학습하여, 학습 데이터와 유사한 새로운 데이터를 만드는 모델
 - 기존 데이터를 그대로 복사하는 것이 아니라, 학습한 특성을 바탕으로 새로운 샘플을 생성
 - 예: 개와 고양이 이미지의 특징을 학습하여 새로운 개·고양이 이미지 생성
- **판별 모델**
 - 입력 데이터가 주어졌을 때 해당 데이터의 정답이나 범주를 예측하는 모델
 - 서로 다른 범주를 구분하는 데 필요한 특징을 학습
 - 예: 이미지 속 동물이 개인지 고양이인지 분류
 - 귀 모양, 눈과 코의 형태, 꼬리 등 분류에 도움이 되는 특징을 활용
- **생성 모델과 판별 모델의 차이**
 - 판별 모델: 주어진 데이터가 무엇인지 판단
 - 생성 모델: 데이터가 어떤 특성과 분포를 가지는지 학습하여 새로운 데이터 생성
 - 교재에서는 같은 이미지 특징을 '분류'와 '생성'이라는 서로 다른 목적에 활용하는 것으로 설명

13.1.2. 생성 모델의 유형

▼ 그림 13-2 생성 모델의 유형



- **명시적 방법**

- 데이터의 확률분포를 명시적으로 정의하여 모델링하는 방법
- 대표적인 모델: **변형 오토인코더(VAE)**
- 잠재변수의 분포를 이용해 샘플을 얻고, 이를 바탕으로 데이터를 생성
- 확률모형을 정의하더라도 데이터의 확률을 항상 정확하고 간단하게 계산할 수 있다는 뜻은 아님

- **암시적 방법**

- 데이터의 확률밀도 함수를 직접 계산하는 대신, 샘플을 생성하는 과정을 학습하는 방법
- 대표적인 모델: **GAN**
- 생성자와 판별자가 경쟁하면서 실제와 유사한 데이터를 생성하도록 학습
- 교재의 '확률 변수를 이용하지 않는다'는 표현은 주의가 필요
- GAN도 일반적으로 무작위 잠재벡터를 사용하며, 핵심 차이는 **데이터의 확률밀도를 명시적으로 다루는지 여부**

- **샘플링**

- 특정 확률분포에 따라 값을 추출하는 과정
- 생성 모델에서는 잠재변수 등을 샘플링하여 다양한 출력을 얻음

- **잠재공간**

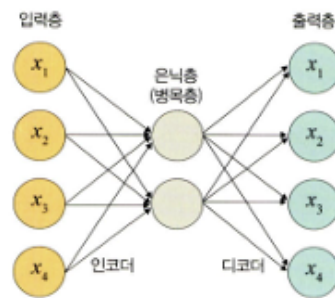
- 데이터의 중요한 특성을 압축하여 표현하는 공간
- 고차원 이미지가 상대적으로 적은 수의 값으로 표현될 수 있음
- 잠재공간에 위치한 개별 표현을 잠재벡터라고 함
- 잠재벡터를 디코더에 입력하면 그 표현에 대응하는 데이터를 얻을 수 있음

13.2. 변형 오토인코더

13.2.1. 오토인코더란

1. 오토인코더의 기본 개념

▼ 그림 13-3 오토인코더



• 오토인코더(Autoencoder, AE)

- 입력 데이터를 압축한 뒤, 원래 입력과 최대한 비슷하게 복원하도록 학습하는 신경망
- 입력 자체가 복원의 목표가 되므로 별도의 분류 정답이 필요하지 않음
- 교재에서는 입력보다 작은 병목층을 사용하는 구조로 설명

• 입력을 다시 출력하는 이유

- 제한된 크기의 잠재표현만으로 원본을 복원해야 하기 때문
- 복원에 필요한 중요한 정보를 압축된 표현에 담도록 유도
- 이를 통해 데이터의 주요 특성을 학습

• 학습 목표

- 입력과 재구성된 출력의 차이를 줄이는 것
- 입력과 출력의 형태 및 차원은 같도록 구성
- 입력을 정확히 복원하는 데 필요한 표현을 인코더와 디코더가 함께 학습

2. 오토인코더의 구성 요소

• 인코더

- 입력 데이터의 특징을 추출하고 압축하는 부분
- 교재에서는 인지 네트워크라고도 설명
- 고차원 입력을 저차원 잠재벡터로 변환

• 병목층

- 입력의 압축된 표현이 위치하는 부분
- 교재의 구조에서는 가장 적은 수의 뉴런을 가진 계층

- 제한된 공간에 원본 복원에 필요한 정보를 담는 역할
- 잠재차원이 지나치게 작으면 복원에 필요한 정보가 손실될 수 있음

• 디코더

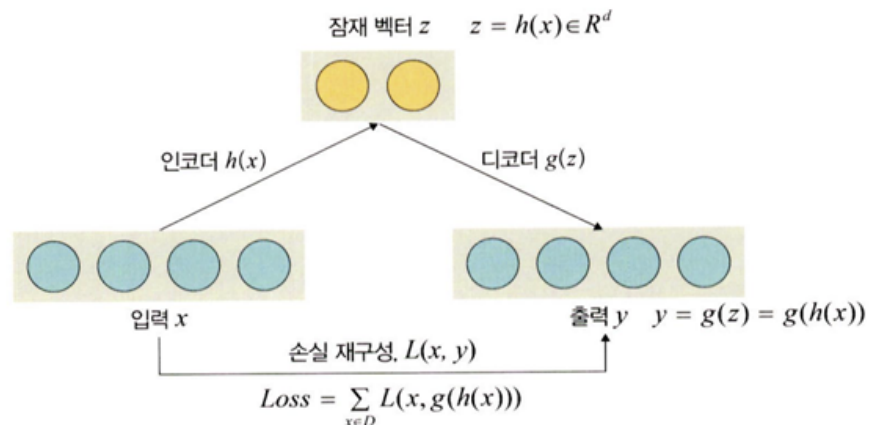
- 압축된 잠재벡터로부터 원래 형태의 데이터를 재구성하는 부분
- 교재에서는 생성 네트워크라고도 설명
- 압축된 정보를 이용하여 입력과 유사한 출력 생성

• 재구성 손실

- 입력과 디코더 출력 사이의 차이를 측정
- 이 차이가 작아지는 방향으로 인코더와 디코더를 함께 학습
- 교재에서 '손실 재구성'으로 표현한 항목에 해당

3. 오토인코더의 활용 목적

▼ 그림 13-4 오토인코더의 인코더와 디코더



• 데이터 압축

- 데이터의 중요한 정보를 상대적으로 적은 수의 값으로 표현
- 이미지나 음성 등의 표현 크기를 줄이는 데 활용 가능
- 압축 정도에 따라 원본 정보가 일부 손실될 수 있음

• 차원 축소

- 많은 특성을 가진 데이터를 더 적은 차원으로 표현
- 고차원 데이터의 처리 부담과 차원의 저주 문제를 완화하는 데 도움
- 차원을 줄였다는 사실만으로 모든 고차원 문제가 해결되는 것은 아님

- **특성 추출**

- 별도의 클래스 정답 없이 데이터의 주요 특성을 학습
- 학습된 잠재벡터를 다른 분석이나 학습의 입력으로 활용 가능

4. MNIST 오토인코더 실습의 전체 흐름

- **실습 목적**

- MNIST 숫자 이미지를 압축하고 다시 재구성
- 이미지에 노이즈를 추가한 뒤 재구성 결과를 관찰
- 학습이 진행되면서 손실과 이미지 품질이 어떻게 달라지는지 확인

- **데이터 준비**

- `torchvision.datasets.MNIST` 로 학습·테스트 데이터 불러오기
- `ToTensor` 로 이미지를 텐서로 변환
- `DataLoader` 로 데이터를 미니배치 단위로 공급
- 숫자의 클래스 레이블은 재구성 학습에 사용하지 않음

- **DataLoader의 주요 설정**

- `batch_size` : 한 번에 처리하는 데이터 개수
- `shuffle` : 데이터를 무작위로 섞을지 결정
- `num_workers` : 데이터를 불러오는 작업에 사용할 프로세스 수
- `pin_memory` : CPU 메모리를 고정하여 GPU 전송을 빠르게 하는 데 활용
- `pin_memory` 는 데이터를 GPU에 미리 저장한다는 의미는 아님

5. 교재의 오토인코더 네트워크 구조

- **인코더**

- 합성곱 계층으로 이미지 특징 추출
- stride를 이용해 이미지의 공간 크기를 점차 축소
- ReLU로 비선형성을 추가하고 일부 계층에 Batch Normalization 적용
- Flatten으로 특징맵을 벡터 형태로 변환
- 완전연결층을 거쳐 최종적으로 **4차원 잠재벡터** 생성

- **디코더**

- 4차원 잠재벡터를 완전연결층으로 확장
- Unflatten으로 벡터를 특징맵 형태로 변환
- 전치 합성곱 계층으로 공간 크기를 점차 확대
- 원래 MNIST 이미지와 같은 크기로 재구성
- 마지막 sigmoid로 출력값을 0~1 범위로 제한

• 주요 연산의 역할

- **Flatten** : 특징맵의 모양을 벡터로 바꾸는 연산
- **Linear** : 학습 가능한 가중치를 가진 완전연결층
- **Unflatten** : 벡터를 지정된 다차원 모양으로 바꾸는 연산
- **ConvTranspose2d** : 학습 가능한 연산으로 공간 크기를 확대하는 데 사용
- Flatten 자체가 완전연결층은 아니며, 전치 합성곱도 앞선 합성곱을 정확히 되돌리는 역연산은 아님

• 인코더와 디코더의 관계

- 인코더의 출력이 디코더의 입력이 됨
- 서로 연결되는 잠재표현의 크기가 맞아야 함
- 최종 출력은 원본과 같은 형태여야 함
- 두 네트워크의 계층 수나 구조가 반드시 동일해야 하는 것은 아님

6. 손실함수와 최적화

• 재구성 손실

- 입력 이미지와 재구성 이미지의 차이를 측정
- 해당 오토인코더 예제에서는 **MSELoss** 사용

• 손실함수 선택

- MSE: 예측값과 목표값 사이의 수치적 차이를 평가
- 이진 교차 엔트로피: 출력값을 픽셀별 확률로 해석하는 경우 등에 활용
- 입력이 0~1 범위라는 이유만으로 반드시 이진 교차 엔트로피를 사용해야 하는 것은 아님

• 최적화 설정

- 옵티마이저: Adam

- 학습률: 0.001
- Weight decay: 0.00001
- 인코더와 디코더의 파라미터를 모두 옵티마이저에 등록

• 학습 과정

- 이미지를 인코더에 입력
- 잠재벡터를 디코더에 전달
- 재구성 손실 계산
- 이전 기울기 초기화
- 역전파로 기울기 계산
- 옵티마이저로 두 네트워크의 파라미터 업데이트

7. 노이즈 추가와 평가

• 노이즈 추가

- 원본 이미지와 같은 크기의 정규분포 잡음 생성
- `noise_factor` 로 잡음의 세기 조절
- 잡음이 더해진 픽셀값을 0~1 범위로 제한
- 교재 예제의 노이즈 세기: 0.3

• 평가 과정

- 인코더와 디코더를 평가 모드로 전환
- 기울기 계산 없이 재구성 결과와 손실 확인
- 평가 중에는 파라미터를 업데이트하지 않음

• 이미지 시각화

- 원본 이미지
- 노이즈가 추가된 이미지
- 오토인코더로 재구성한 이미지
- 세 종류를 함께 배치하여 재구성 결과 비교

8. 오토인코더 학습 결과

• 학습 설정

- 총 30에포크 학습
- 에포크별 학습 손실, 검증 손실, 재구성 이미지 확인
- **교재에 제시된 결과**
 - 1에포크: 학습 손실 0.089, 검증 손실 0.050
 - 12에포크: 학습 손실 0.050, 검증 손실 0.039
 - 30에포크: 학습 손실 0.049, 검증 손실 0.037
- **결과 해석**
 - 학습이 진행되면서 재구성 오차가 감소
 - 초기보다 숫자의 형태가 더 분명하게 나타남
 - 압축된 잠재표현으로 원본의 주요 형태를 복원하는 능력이 개선됨

▼ 그림 13-6 첫 번째 에포크에 대한 이미지 결과



▼ 그림 13-7 12번째 에포크에 대한 이미지 결과



... 중간 생략 ...

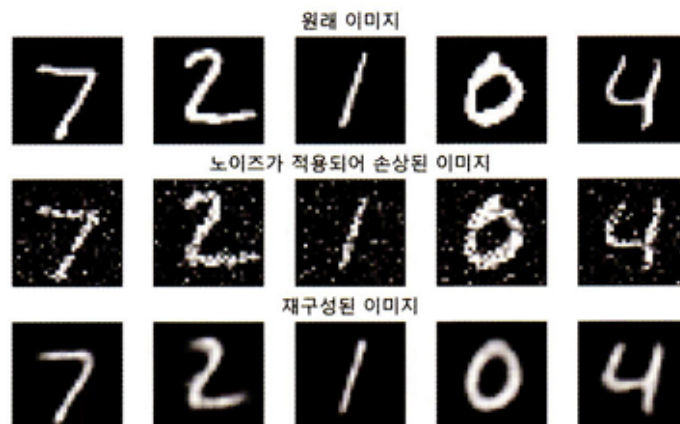
EPOCH 30/30

EPOCH 30/30

train loss 0.049

val loss 0.037

▼ 그림 13-8 30번째 에포크에 대한 이미지 결과



13.2.2. 변형 오토인코더 — p.695~708

1. 일반 오토인코더에서 VAE로 확장

• 일반 오토인코더

- 입력을 하나의 잠재벡터로 압축
- 해당 잠재벡터를 이용해 입력을 재구성
- 기본적인 재구성 학습만으로는 잠재공간이 특정 확률분포를 따르도록 보장되지 않음
- 임의의 잠재벡터를 선택했을 때 의미 있는 데이터가 생성된다는 보장도 부족

- **변형 오토인코더**

- Variational Autoencoder, VAE
- 일반적으로 '변분 오토인코더'라고도 부름
- 하나의 잠재벡터 대신 **잠재변수의 확률분포를 표현하는 파라미터**를 학습
- 그 분포에서 잠재벡터를 샘플링하여 디코더에 입력
- 재구성과 함께 잠재분포에 대한 정규화를 수행

- **핵심 차이**

- AE: 입력을 압축한 특정 표현을 이용
- VAE: 입력에 대응하는 잠재분포를 학습하고 그 분포에서 샘플링
- VAE는 잠재공간을 확률적으로 구성하여 다양한 데이터 생성에 활용

2. VAE의 인코더와 잠재분포

- **인코더의 역할**

- 입력 데이터에 대응하는 잠재분포를 추정
- 교재에서는 Gaussian 분포를 사용
- 분포의 평균과 분산에 해당하는 정보를 출력

- **평균**

- 잠재표현이 어느 위치를 중심으로 분포하는지 나타냄

- **분산과 표준편차**

- 잠재표현이 중심 주변으로 얼마나 퍼져 있는지 나타냄
- 잠재변수별 샘플링의 변동 폭에 관여

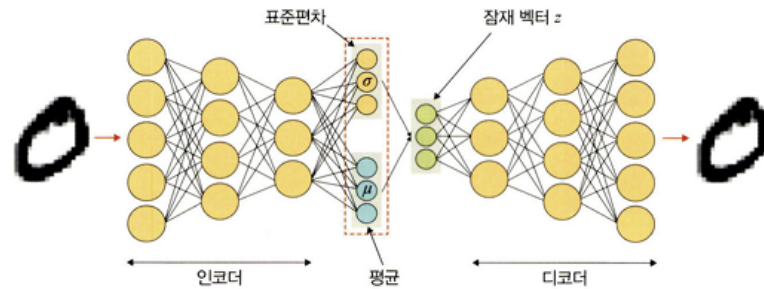
- **잠재벡터 샘플링**

- 인코더가 예측한 분포에서 하나의 잠재벡터를 추출
- 추출된 잠재벡터를 디코더에 전달
- 같은 입력에서도 샘플링 결과에 따라 재구성이 달라질 수 있음

- **잠재분포를 학습하는 이유**

- 개별 입력을 복원하는 표현을 얻는 동시에, 생성에 활용할 수 있는 확률적 구조를 형성하기 위해서

▼ 그림 13-10 변형 오토인코더 실행 과정



3. VAE의 디코더

• 디코더의 역할

- 샘플링한 잠재벡터를 입력받아 데이터의 분포 또는 재구성 결과를 출력
- 잠재변수에 담긴 정보를 이미지 공간으로 변환
- 학습 중에는 원본 입력을 잘 설명하는 출력을 만들도록 조정

• 개념 설명과 실습 코드의 차이

- 교재의 개념도는 Gaussian 출력분포의 평균과 분산으로 디코더를 설명
- 실제 MNIST 실습 코드는 sigmoid 출력과 이진 교차 엔트로피를 사용
- 실습에서는 픽셀별 Bernoulli 확률로 해석할 수 있는 출력값을 예측

• 재구성과 새로운 생성의 구분

- 재구성: 관측 이미지를 인코더에 넣고, 그 결과로 얻은 잠재벡터를 디코더에 전달
- 새로운 생성: 학습 후 사전분포에서 잠재벡터를 샘플링하여 디코더에 전달
- 교재 실습의 평가 코드는 주로 입력 이미지와 재구성 이미지를 비교

4. VAE의 학습 목적

• 재구성 성능

- 잠재벡터로부터 원본 데이터를 잘 복원하도록 학습
- 원본 데이터에 대한 디코더의 로그우도를 높이는 방향
- 손실 최소화 관점에서는 재구성 오차를 줄이는 것에 해당

• 잠재분포 정규화

- 입력별 잠재분포가 기준이 되는 사전분포에서 지나치게 벗어나지 않도록 학습
- 교재 실습에서는 표준정규분포를 기준으로 사용

- 두 분포의 차이를 KL divergence로 측정
- **두 목표를 함께 사용하는 이유**
 - 재구성만 강조하면 생성에 적합한 잠재분포가 형성되지 않을 수 있음
 - 사전분포와의 일치만 강조하면 입력을 복원하는 데 필요한 정보가 부족해질 수 있음
 - 원본 복원과 잠재분포 정규화 사이의 균형이 필요
- **ELBO와의 연결**
 - 교재의 목적함수는 재구성 로그우도를 높이고 KL divergence를 줄이는 구조
 - 이는 VAE의 변분 하한인 ELBO를 최대화하는 관점
 - 실습에서는 부호를 바꿔 재구성 손실과 KL 손실을 합한 값을 최소화

5. 변분추론

- **변분추론의 의미**
 - 직접 구하기 어려운 실제 사후분포를 다루기 쉬운 분포로 근사하는 방법
 - Gaussian과 같은 분포 형태를 선택
 - 해당 분포의 파라미터를 조정하면서 실제 사후분포를 근사
- **VAE에서의 역할**
 - 입력을 설명하는 잠재변수의 사후분포를 인코더로 근사
 - 인코더는 입력마다 평균과 분산 정보를 예측
 - 생성모형과 근사 추론모형을 함께 학습
- **구분할 점**
 - 실제 사후분포를 근사한다는 변분추론의 목표와, 손실함수의 KL 항은 관련되지만 동일한 표현은 아님
 - 실습에서 직접 계산하는 KL 항은 **인코더의 근사 사후분포와 표준정규 사전분포 사이의 차이**

6. MNIST 기반 VAE 구현

- **데이터 준비**
 - MNIST 이미지를 텐서로 변환
 - 학습 및 테스트 배치 크기: 100

- 28×28 이미지를 길이 784의 벡터로 펼쳐 완전연결층에 입력
- **인코더 구조**
 - 입력 차원: 784
 - 은닉 차원: 400
 - 두 개의 은닉층에서 LeakyReLU 사용
 - 마지막에 평균과 로그 분산을 출력하는 두 갈래로 분리
 - 각 출력의 차원: 200
- **디코더 구조**
 - 잠재벡터 차원: 200
 - 은닉 차원: 400
 - 두 개의 은닉층에서 LeakyReLU 사용
 - 최종 출력 차원: 784
 - 마지막 sigmoid로 출력값을 0~1 범위로 제한
- **일반 AE 실습과의 차이**
 - 앞선 AE는 합성곱 기반 구조와 4차원 잠재벡터 사용
 - VAE는 완전연결층 기반 구조와 200차원 잠재변수 사용
 - 따라서 두 예제의 성능 차이를 AE와 VAE의 학습 원리 차이만으로 해석할 수는 없음

7. 로그 분산과 재매개변수화

- **로그 분산을 출력하는 이유**
 - 분산은 양수여야 하지만 일반적인 신경망 출력은 음수가 될 수도 있음
 - 제약 없는 로그 분산을 예측한 뒤 지수 변환하여 양수인 분산이나 표준편차를 얻음
 - 로그 분산 자체는 음수일 수 있음
- **교재 코드의 `log_var`**
 - 로그를 취한 분산을 의미
 - 표준편차나 로그 표준편차와 구분해야 함
 - 실제 샘플링 전에는 로그 분산을 표준편차로 변환
- **재매개변수화**

- 표준정규분포에서 잡음을 샘플링
- 예측한 표준편차로 잡음의 크기를 조절
- 예측한 평균을 더해 잠재벡터 생성
- 원래 Gaussian 잠재분포에서 샘플링하는 것과 같은 분포를 얻음
- **재매개변수화가 필요한 이유**
 - 무작위 샘플링을 파라미터와 독립적인 잡음으로 분리
 - 평균과 표준편차에서 잠재벡터로 이어지는 연산을 미분 가능하게 구성
 - 디코더의 손실을 인코더까지 역전파할 수 있게 함
- **코드 해석 시 주의**
 - 교재의 재매개변수화 함수에서 인자 이름은 `var`
 - 실제 전달되는 값은 분산이 아니라 **표준편차**
 - 변수 이름보다 호출 시 수행하는 변환을 기준으로 이해해야 함

8. VAE 손실함수

$$L = \underbrace{-E_{z \sim q(z|x)}[\log p_\theta(x|z)]}_{\text{a)}} + \underbrace{D_{KL}(q(z|x) \parallel p_\theta(z))}_{\text{b)}}$$

- **재구성 손실**
 - 원본 이미지와 재구성 이미지의 차이를 평가
 - 교재 실습에서는 이진 교차 엔트로피 사용
 - 픽셀과 배치에 걸친 손실을 합산하여 계산
- **KL divergence**
 - 인코더의 Gaussian 근사분포와 표준정규 사전분포 사이의 차이를 측정
 - 인코더가 출력한 평균과 로그 분산을 이용해 계산
 - 잠재분포가 사전분포에서 지나치게 벗어나지 않도록 정규화
- **최종 손실**
 - 재구성 손실과 KL 손실을 더해 사용
 - 두 항의 합이 줄어드는 방향으로 학습

- KL만 최소화하거나 재구성 손실만 최소화하는 방식과 구분

- **최적화 설정**

- 옵티마이저: Adam
- 학습률: 0.001
- 학습 에포크: 30

9. 학습 및 평가

- **학습 과정**

- 입력 이미지를 벡터로 변환
- 인코더가 평균과 로그 분산 계산
- 재매개변수화로 잠재벡터 샘플링
- 디코더가 이미지 재구성
- 재구성 손실과 KL 손실 계산
- 전체 손실을 역전파하여 파라미터 업데이트

- **평가 과정**

- 평가 모드에서 기울기 계산 없이 실행
- 테스트 데이터의 재구성 손실, KL 손실, 전체 손실 계산
- 원본과 재구성 이미지를 함께 기록

- **TensorBoard의 역할**

- 학습 과정에서 측정값이 어떻게 변하는지 시각적으로 확인
- 학습과 테스트 결과를 구분하여 기록
- 전체 손실뿐 아니라 구성 요소도 따로 관찰

- **기록하는 항목**

- Reconstruction Error: 재구성 오차
- KL Divergence: 잠재분포 정규화 항
- Total Loss: 두 항을 합한 전체 손실
- 원본 이미지와 재구성 이미지

- **주요 기능**

- `SummaryWriter`: 로그 기록 객체 생성
- `add_scalar`: 손실 등의 수치 기록
- `add_image`: 이미지 기록
- `close`: 남은 로그를 저장하고 기록 종료
- **재구성 오차**
 - 초기에 크게 감소한 뒤 점차 안정화
 - 입력 이미지의 특성을 복원하는 능력이 개선되는 것으로 해석
- **KL divergence**
 - 재구성 오차와 달리 학습 중 증가한 뒤 일정 범위에서 변동하는 모습이 나타남
 - 잠재표현이 입력에 대한 정보를 담으면서 사전분포와 일정한 차이를 가질 수 있음
 - KL이 증가했다는 사실만으로 학습 실패라고 판단하지 않음
- **전체 손실**
 - 재구성 오차와 KL 손실의 합
 - 한 항이 증가하더라도 다른 항이 충분히 감소하면 전체 손실은 감소할 수 있음
 - 두 항을 함께 살펴보며 학습 상태를 판단해야 함